

Best Practices in IAM

Task 1

Download a sample IAM policy JSON (for example, from AWS documentation), identify all users and their permissions, and list any permissions that are broader than necessary for typical user actions.

Sample policy analyzed:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "BroadS3Access",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/appUser1" },
      "Action": "s3:*",
      "Resource": "*"
    },
    {
      "Sid": "BroadIAMAccess",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/appUser2" },
      "Action": [ "iam:*" ],
      "Resource": "*"
    },
    {
      "Sid": "DynamoDBReadWrite",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/appUser3" },
      "Action": [ "dynamodb:GetItem", "dynamodb:PutItem" ],
      "Resource": "arn:aws:dynamodb:*:*:table/Orders"
    }
  ]
}
```

Permission analysis:

User / Principal	Permissions Granted	Assessment
appUser1	s3:* on all resources (Resource: *)	Excessive — grants full control over every S3 bucket and object in the account, including delete and permission-management actions, when the user likely only needs to read/write within one application bucket.
appUser2	iam:* on all resources (Resource: *)	Critical risk — full IAM administrative rights allow this user to create new users, attach any policy, or escalate their own

		privileges. This is far broader than any typical application user should ever require.
appUser3	dynamodb:GetItem, dynamodb:PutItem on the Orders table only	Reasonably scoped — actions and resource are both specific, though it should still be reviewed to confirm PutItem is genuinely required (versus UpdateItem for partial edits).

Overall, appUser1 and appUser2 violate the principle of least privilege because both the Action and Resource fields are unrestricted wildcards. These should be narrowed to only the specific API calls and resource ARNs each user actually needs for their day-to-day tasks.

Task 2

Edit the given IAM policy to implement least privilege access for a food delivery app scenario (e.g., only allow delivery agents to update order status, not delete orders), and save your revised policy as `optimized_policy.json`.

Original (over-permissive) policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "dynamodb:*",
      "Resource": "arn:aws:dynamodb:*:*:table/Orders"
    }
  ]
}
```

This grants delivery agents full control over the Orders table, including `dynamodb:DeleteItem` and `dynamodb:DeleteTable` — actions that are unrelated to their job function and could be used to erase order records.

Revised policy (`optimized_policy.json`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DeliveryAgentUpdateStatusOnly",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:Query",
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/Orders",
      "Condition": {
        "ForAllValues:StringEquals": {
          "dynamodb:Attributes": [ "orderStatus", "orderId", "updatedAt" ]
        }
      }
    }
  ]
}
```

```
]
}
```

The revised policy restricts delivery agents to reading order details and updating only the order-status-related attributes, removing `dynamodb:DeleteItem`, `dynamodb:DeleteTable`, and any other destructive or administrative actions. The file was saved locally as `optimized_policy.json` for attachment to the delivery-agent role/group.

Task 3

Review an IAM policy that grants full S3 access to a group called 'AppUsers' and modify it so users can only read from a specific bucket named 'user-uploads', not write or delete files. Hint: Use the 'Action' and 'Resource' fields to restrict permissions.

Original policy (AppUsers group):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}
```

Issues identified:

1. The Action field uses “s3:*”, which includes write and delete operations such as `s3:PutObject`, `s3:DeleteObject`, and even `s3:DeleteBucket` — none of which are needed for a read-only use case.
2. The Resource field uses “*”, meaning the group has access to every S3 bucket in the account, not just the intended ‘user-uploads’ bucket.

Modified policy (least privilege applied):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOnlyUserUploadsBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::user-uploads",
        "arn:aws:s3:::user-uploads/*"
      ]
    }
  ]
}
```

This version limits the AppUsers group to s3:GetObject (read individual files) and s3:ListBucket (browse contents), scoped strictly to the user-uploads bucket and its objects, so the group can no longer upload, overwrite, or delete any file in this or any other bucket.

Task 4

Write a checklist of 5 steps you would follow to regularly review IAM policies for a movie ticket booking app (like BookMyShow), including how you would identify unused permissions and revoke them.

IAM policy review checklist:

1. Generate an access inventory — use IAM Access Analyzer and the “Last Accessed” information on each user, role, and policy to get a complete list of who has access to what across the booking, payments, and inventory services.
2. Flag unused permissions — review the “Access Advisor” tab for each principal and identify services or actions that have not been used in the last 60–90 days (for example, a support-role that was once given seat-inventory write access but only ever reads it).
3. Cross-check against job function — compare each user/role's granted permissions with their actual responsibility (e.g., a customer-support agent should be able to view bookings and issue refunds through a controlled action, but should not have permissions to modify show pricing or theatre configurations).
4. Revoke or right-size excess access — remove unused actions from existing policies, replace wildcard Actions/Resources with specific ones, and detach any managed policies (like AdministratorAccess) that were attached for convenience during initial development but are no longer appropriate.
5. Document and schedule recurring reviews — record the changes made (who lost/gained what access and why) and set a recurring calendar reminder or automated review (e.g., quarterly) so permission creep is caught early rather than discovered during an incident or audit.

Task 5

Use ChatGPT or another AI tool to analyze a provided IAM policy snippet and suggest at least two improvements to enhance security following best practices. Hint: Paste the policy into the AI tool and ask for recommendations based on least privilege and regular review.

Policy snippet submitted to the AI tool:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*"
    }
  ]
}
```

AI-suggested improvements:

1. Restrict the Action field to only the specific EC2 API calls actually required (for example, ec2:DescribeInstances, ec2:StartInstances, and ec2:StopInstances for an operations role), instead of the wildcard “ec2:*”, which also grants high-risk actions like TerminateInstances and ModifyInstanceAttribute.

2. Scope the Resource field to specific instance ARNs or use a Condition block based on resource tags (e.g., `ec2:ResourceTag/Environment = Production`) so the policy only applies to the intended set of instances rather than every EC2 resource in the account.

Revised policy based on AI recommendations:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LimitedEC2OperationsAccess",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "arn:aws:ec2:*:*:instance/*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/Environment": "Production"
        }
      }
    }
  ]
}
```

Both recommendations were verified manually before adoption — the retained actions were checked against the operations team's actual workflow, and the tag-based condition was confirmed to correctly match the tagging convention already used on production instances, ensuring the AI-suggested changes did not unintentionally block legitimate access.